

EXAMEN DE PROGRAMACIÓN – 1º GRADO SUPERIOR

Examen 15 · Ordenación y búsqueda: algoritmos y complejidad

Algoritmos básicos: burbuja, selección e inserción

Arrays.sort y Collections.sort

Comparable y Comparator (incluidas lambdas)

Búsqueda binaria: requisitos e implementación

Complejidad algorítmica: $O(n)$, $O(n^2)$, $O(n \log n)$

Asignatura	Programación (PRG)
Curso	1º Grado Superior — Desarrollo de Aplicaciones Web (DAW)
Duración	2 horas
Puntuación total	16 puntos
Convocatoria	Evaluación 3
Valoración	Algoritmos de ordenación y búsqueda

Parte 1: Opción múltiple y Verdadero/Falso

5 puntos — 0,5 cada pregunta

1. ¿Qué complejidad temporal tiene la ordenación por burbuja en el peor caso?
- a) $O(n^2)$
 - b) $O(n \log n)$
 - c) $O(n)$
 - d) $O(\log n)$

RESPUESTA / SOLUCIÓN

a) $O(n^2)$ — Dos bucles anidados sobre n elementos; lo mismo aplica a selección e inserción en el peor caso.

-
2. ¿Qué método de la clase Arrays ordena un array?
- a) `Arrays.sort()`
 - b) `Arrays.order()`
 - c) `Arrays.sortArray()`
 - d) `Arrays.quickSort()`

RESPUESTA / SOLUCIÓN

a) `Arrays.sort()` — Ordena arrays de primitivos o de objetos (con Comparable o Comparator).

-
3. ¿Qué requisito imprescindible tiene la búsqueda binaria?
- a) El array debe estar ordenado
 - b) El array no puede tener duplicados

- c) Solo funciona con enteros
- d) El array debe ser pequeño

RESPUESTA / SOLUCIÓN

- a) Estar ordenado — Divide el espacio en dos en cada paso; sin orden previo las comparaciones con el medio no garantizan nada.

-
4. ¿Qué interfaz deben implementar los objetos para que `Arrays.sort` los ordene con su orden natural?
- a) Comparable
 - b) Comparator
 - c) Orderable
 - d) Sortable

RESPUESTA / SOLUCIÓN

- a) Comparable — Define `compareTo(T otro)`; p. ej. los `String` ya son `Comparable`.

-
5. ¿Qué método define la interfaz `Comparable`?
- a) `compareTo(T o)`
 - b) `compare(T a, T b)`
 - c) `compareTo(T a, T b)`
 - d) `equals(T o)`

RESPUESTA / SOLUCIÓN

a) `compareTo(T o)` — Devuelve negativo, cero o positivo según este objeto sea menor, igual o mayor que el argumento.

6. ¿Qué interfaz permite ordenar una colección con criterios externos sin tocar la clase?

- a) `Comparator`
- b) `Comparable`
- c) `Orderable`
- d) `Sortable`

RESPUESTA / SOLUCIÓN

a) `Comparator` — Con `compare(a, b)`; se puede pasar a `Arrays.sort/Collections.sort` y combinarse con lambdas o `comparing()`.

7. ¿Qué método de `Collections` ordena una `List`?

- a) `Collections.sort(lista)`
- b) `Collections.order(lista)`
- c) `Collections.sortList(lista)`
- d) `Arrays.sort(lista)`

RESPUESTA / SOLUCIÓN

a) `Collections.sort(lista)` — Ordena la lista in-place; acepta un `Comparator` opcional como segundo argumento.

-
8. **Aplicar búsqueda binaria a un array desordenado puede devolver resultados incorrectos sin lanzar ninguna excepción.**

RESPUESTA / SOLUCIÓN

Verdadero — El algoritmo asume orden y no lo comprueba: puede devolver -1 o una posición equivocada silenciosamente.

-
9. **¿Qué algoritmo usa internamente `Arrays.sort` para objetos en Java moderno (caso promedio $O(n \log n)$)?**
- a) TimSort (variante de merge sort)
 - b) Burbuja
 - c) Selección
 - d) Inserción

RESPUESTA / SOLUCIÓN

a) TimSort — Ordenación estable $O(n \log n)$ en el caso promedio; para primitivos usa dual-pivot quicksort.

10. En la búsqueda binaria, si el valor buscado es MAYOR que el elemento central, ¿en qué mitad se continúa?

- a) En la mitad derecha
- b) En la mitad izquierda
- c) Se termina
- d) Se reinicia el algoritmo

RESPUESTA / SOLUCIÓN

a) Mitad derecha — El elemento central descarta toda la mitad izquierda (incluido él mismo).

Parte 2: Análisis y depuración

3 puntos

1. Este programa busca un valor y a veces dice que no existe aunque sí está. ¿Por qué? (1,5 pts)

```
int[] datos = {12, 5, 8, 23, 16, 2};
int buscado = 16;
int pos = Arrays.binarySearch(datos, buscado);
System.out.println(pos >= 0 ? "Encontrado en " + pos : "No existe");
```

RESPUESTA / SOLUCIÓN

Arrays.binarySearch exige un array ordenado; como datos no está ordenado, el resultado es impredecible (puede devolver -1 aunque 16 esté presente).

Corrección: ordenar antes: Arrays.sort(datos); y luego buscar. Alternativa: búsqueda lineal si no se puede ordenar.

-
2. ¿Por qué este código no compila y cómo se arregla de dos formas distintas? (1,5 pts)

```
class Persona {
    String nombre;
    int edad;
    Persona(String n, int e) { nombre = n; edad = e; }
}
// ...
List<Persona> lista = new ArrayList<>();
lista.add(new Persona("Ana", 25));
Collections.sort(lista);
```

RESPUESTA / SOLUCIÓN

`Collections.sort(lista)` exige que `Persona` implemente `Comparable<Persona>`; si no, el compilador falla.

Solución 1: que `Persona` implemente `Comparable<Persona>` con `compareTo` por edad (o nombre).

Solución 2: pasar un `Comparator`: `Collections.sort(lista, Comparator.comparingInt(p -> p.edad))`;

Parte 3: Ejercicios de programación

6 puntos

1. Implementa a mano la ordenación por selección: en cada pasada encuentra el mínimo de la parte no ordenada y lo coloca en su sitio. Muéstralo con un array de 5 enteros. (2 ptos)

RESPUESTA / SOLUCIÓN

```
public class Seleccion {  
    public static void ordenar(int[] a) {  
        for (int i = 0; i < a.length - 1; i++) {  
            int min = i;  
            for (int j = i + 1; j < a.length; j++) {  
                if (a[j] < a[min]) min = j;  
            }  
            int aux = a[i]; a[i] = a[min]; a[min] = aux;  
        }  
    }  
    public static void main(String[] args) {  
        int[] a = {29, 10, 14, 37, 13};  
        ordenar(a);  
        System.out.println(java.util.Arrays.toString(a)); // [10, 13, 14, 29, 37]  
    }  
}
```

2. Ordena una lista de alumnos (nombre, nota) por nota descendente y, a igual nota, por nombre ascendente, usando Comparator con lambdas. (2 ptos)

RESPUESTA / SOLUCIÓN

```
import java.util.*;

class Alumno {
    String nombre; double nota;

    Alumno(String n, double nt) { nombre = n; nota = nt; }

    public String toString() { return nombre + " (" + nota + ")"; }
}

public class OrdenarAlumnos {
    public static void main(String[] args) {
        List<Alumno> lista = new ArrayList<>(List.of(
            new Alumno("Ana", 7.5), new Alumno("Luis", 9.0), new Alumno("Eva", 7.5)));
        lista.sort(Comparator.comparingDouble((Alumno a) -> a.nota).reversed()
            .thenComparing(a -> a.nombre));

        System.out.println(lista); // [Luis (9.0), Ana (7.5), Eva (7.5)]
    }
}
```

-
- 3. Implementa la búsqueda binaria de forma recursiva (método buscar(int[] a, int objetivo, int ini, int fin)) y pruébala con un array ordenado. (2 pts)**

RESPUESTA / SOLUCIÓN

```
public class BusquedaBinaria {  
    static int buscar(int[] a, int objetivo, int ini, int fin) {  
        if (ini > fin) return -1;  
        int medio = (ini + fin) / 2;  
        if (a[medio] == objetivo) return medio;  
        if (objetivo < a[medio]) return buscar(a, objetivo, ini, medio - 1);  
        return buscar(a, objetivo, medio + 1, fin);  
    }  
    public static void main(String[] args) {  
        int[] datos = {2, 5, 8, 12, 16, 23};  
        System.out.println(buscar(datos, 16, 0, datos.length - 1)); // 4  
        System.out.println(buscar(datos, 7, 0, datos.length - 1)), // -1  
    }  
}
```

Parte 4: Pregunta teórica

2 puntos

1. Compara burbuja, selección e inserción en cuanto a complejidad y estabilidad. ¿Cuándo conviene usar `Arrays.sort/Collections.sort` en lugar de implementar uno propio?

RESPUESTA / SOLUCIÓN

Los tres son $O(n^2)$ en el peor caso; inserción es $O(n)$ con datos casi ordenados y es estable; selección no es estable; burbuja es estable pero casi siempre la peor opción.

`Arrays.sort/Collections.sort` usan algoritmos $O(n \log n)$ (TimSort/dual-pivot quicksort) y están muy optimizados, así que para datos reales siempre conviene usarlos.

Implementar a mano solo tiene sentido didáctico, con datos muy pequeños, o cuando se necesita un criterio muy concreto de estabilidad o comportamiento.

Plantilla de respuestas

(Páginas en blanco para desarrollar las soluciones)